



Vijay Academy Presents



Savitribai Phule Pune University (SPPU)

Web Technology (WT)

TE Computer Engineering (2019 Pattern)

One Shot Lecture

Unit 6. Ruby and Rails

What You Get : Strategy For Exam :

- Simple Explanation
- PPT + Notes
- PYQs + Most IMP Questions



Subscribe The Vijay Academy For Free Engineering Content

Unit 6. Ruby and Rails

Syllabus Topics :

Ruby Topics:

1. Introduction to Ruby — Origins & Uses
2. Scalar Types and Their Operations in Ruby
3. Simple Input and Output in Ruby
4. Control Statements in Ruby
5. Fundamentals of Arrays in Ruby
6. Hashes in Ruby
7. Methods in Ruby
8. Classes in Ruby
9. Code Blocks and Iterators in Ruby
10. Pattern Matching in Ruby

Rails Topics:

11. Overview of Rails
12. Document Requests in Rails
13. Processing Forms in Rails
14. Rails Applications and Databases
15. Layouts in Rails
16. Rails with Ajax

EJB Topic:

17. Introduction to EJB



1. Introduction to Ruby — Origins & Uses

What is Ruby?

Ruby is a **simple, object oriented programming language**.

Everything in Ruby is an **object** — even numbers and strings.

Think of it like a **very clean and simple version of Java** — easy to read, easy to write.

Origins of Ruby :

Created by **Yukihiro Matsumoto** (called "Matz")

Created in **1995** in **Japan**.

Matz wanted a language that is **fun to program**.

Name "**Ruby**" was chosen as a gemstone name (like Perl)

First public release = **Ruby 1.0** in **1996**.

Uses of Ruby :

Web development — most popular use (using Rails framework)

Scripting — automate tasks

Data processing — process large data

Prototyping — quickly build and test ideas

Command line tools

Used by big companies — **GitHub, Airbnb, Shopify, Twitter (early)**



Features of Ruby :

1. Object Oriented

Everything is an object in Ruby — even numbers!

```
puts 5.class # Integer  
puts "Hi".class # String
```

2. Simple & Clean Syntax

Very easy to read — almost like English.

```
puts "Hello World"
```

3. Dynamic Typing

No need to declare variable type.

```
name = "Vijay" # string  
age = 21 # integer
```

4. Garbage Collection ---- Automatic memory management.

5. Open Source & Free ---- Anyone can use and modify Ruby.

6. Cross Platform ---- Runs on Windows, Linux, Mac.

7. Rich Library ---- Huge collection of ready made libraries called **Gems**.



Ruby Program :

```
# Hello World in Ruby  
puts "Hello World!"  
puts "Welcome to Ruby!"
```

Output:

```
Hello World!  
Welcome to Ruby!
```

Explanation:

puts → print output with new line
→ single line comment
No semicolons & curly braces needed!

2. Scalar Types and Their Operations in Ruby

What are Scalar Types?

Scalar types are **basic data types** that hold a **single value**.

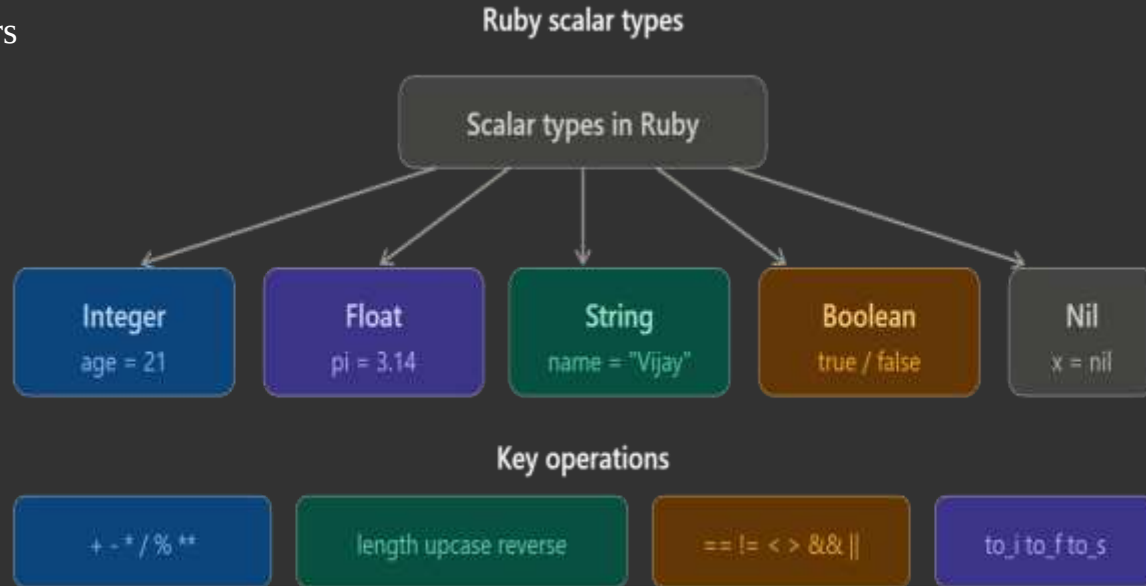
Ruby has 4 main scalar types:

Integer — whole numbers

Float — decimal numbers

String — text

Boolean — true or false.





1. Integer

Whole numbers — no decimal point.

age = 21

marks = 95

year = 2024

```
puts age      # 21
```

```
puts age.class # Integer
```

Integer Operations:

a = 10

b = 3

```
puts a + b    # 13 addition
```

```
puts a - b    # 7 subtraction
```

```
puts a * b    # 30 multiplication
```

```
puts a / b    # 3 division (integer result)
```

```
puts a % b    # 1 modulus (remainder)
```

```
puts a ** b   # 1000 power (10 to the power 3)
```

2. Float

Numbers with decimal point.

price = 99.99

pi = 3.14

```
puts price    # 99.99
```

```
puts price.class # Float
```

Float Operations:

a = 10.5

b = 2.5

```
puts a + b    # 13.0
```

```
puts a - b    # 8.0
```

```
puts a * b    # 26.25
```

```
puts a / b    # 4.2
```



3. String

Text inside single or double quotes.

```
name = "Vijay"
```

```
city = 'Pune'
```

```
puts name      # Vijay
```

```
puts name.class # String
```

String Operations:

```
name = "Vijay"
```

```
puts name.length      # 5
```

```
puts name.upcase      # VIJAY
```

```
puts name.downcase    # vijay
```

```
puts name.reverse     # yajiv
```

```
puts name.include?("ij") # true
```

4. Boolean

Only two values — true or false

```
isPass = true
```

```
isAdmin = false
```

Comparison Operators (give Boolean result):

```
a = 10
```

```
b = 20
```

```
puts a == b  # false (equal)
```

```
puts a != b  # true  (not equal)
```

```
puts a < b   # true  (less than)
```

```
puts a > b   # false (greater than)
```

```
puts a <= b  # true  (less than or equal)
```

```
puts a >= b  # false (greater than or equal)
```


5. Nil

Special type in Ruby — means **no value** / **empty**. Like null in Java or PHP.

```
x = nil
puts x      # (blank)
puts x.class # NilClass
puts x.nil?  # true
```

Type Conversion :

```
puts "21".to_i    # "21" string → 21 integer
puts "3.14".to_f  # "3.14" string → 3.14 float
puts 21.to_s      # 21 integer → "21" string
puts 3.14.to_i    # 3.14 float → 3 integer
```



3. Simple Input and Output in Ruby

Input in Ruby

```
gets — Take input from user
puts "Enter your name:"
name = gets
puts "Hello #{name}"
```

Output:

```
Enter your name:
Vijay
Hello Vijay
```

Output in Ruby

Puts :

Most commonly used. Prints output and **adds new line** automatically.

```
puts "Hello World"
puts "Welcome to Ruby"
puts 42
puts 3.14
```

Output:

```
Hello World
Welcome to Ruby
42
3.14
```

4. Control Statements in Ruby

What are Control Statements?

Control statements **control the flow** of a Ruby program.

They decide **which code runs** and **how many times**.

Same concept as PHP/Java but Ruby syntax is **cleaner and simpler**.

Types of Control Statements

1. if Statement

Run code only if condition is true.

```
marks = 75
```

```
if marks >= 40  
  puts "Pass"  
end
```

Output: Pass



4. Control Statements in Ruby

2. if-else Statement

```
marks = 30
```

```
if marks >= 40
  puts "Pass"
else
  puts "Fail"
end
```

Output: Fail

3. if-elsif-else Statement

Note — Ruby uses **elsif** not **elseif**

```
marks = 85
```

```
if marks >= 75
  puts "Distinction"
elsif marks >= 60
  puts "First Class"
elsif marks >= 40
  puts "Pass"
else
  puts "Fail"
end
```

Output: Distinction



4. unless Statement

Opposite of if — runs when condition is **false**.

Unique to Ruby!

```
marks = 30
```

```
unless marks >= 40
```

```
  puts "You have failed!"
```

```
end
```

Output: You have failed!

5. case Statement

Like switch in PHP/Java. Uses when instead of case.

```
day = 2
```

```
case day
```

```
when 1
```

```
  puts "Monday"
```

```
when 2
```

```
  puts "Tuesday"
```

```
when 3
```

```
  puts "Wednesday"
```

```
else
```

```
  puts "Other day"
```

```
end
```

Output: Tuesday

6. case with Range

```
marks = 85
```

```
case marks
```

```
when 75..100
```

```
  puts "Distinction"
```

```
when 60..74
```

```
  puts "First Class"
```

```
when 40..59
```

```
  puts "Pass"
```

```
else
```

```
  puts "Fail"
```

```
end
```

Output: Distinction

Note — 75..100 is a **range** in Ruby!

7. while Loop

Repeat while condition is true.

```
i = 1
```

```
while i <= 5
```

```
  puts i
```

```
  i += 1
```

```
end
```

Output:

1

2

3

4

5

8. for Loop

```
for i in 1..5  
  puts i  
end
```

Output:

```
1  
2  
3  
4  
5
```

9. times Loop

Very unique to Ruby — repeat N times!

```
5.times do  
  puts "Hello!"  
end
```

Output:

```
Hello!  
Hello!  
Hello!  
Hello!  
Hello!
```



5. Fundamentals of Arrays in Ruby

What is an Array in Ruby?

An array is a **collection of multiple values** stored in one variable.

Values can be of **any type** — integers, strings, mixed.

Ruby arrays are **more flexible** than PHP or Java arrays.

Types of Arrays in Ruby :

Type 1 — Single Dimensional Array

Simple array with one row of elements.

```
fruits = ["Apple", "Mango", "Banana"]
```

```
nums = [1, 2, 3, 4, 5]
```

```
puts fruits[0] # Apple
```

```
puts nums[2] # 3
```




Type 2 — Multi Dimensional Array

Array inside array — like a table.

```
students = [  
  ["Vijay", 21, "Pune"],  
  ["Rahul", 22, "Mumbai"],  
  ["Priya", 20, "Nashik"]  
]  
  
puts students[0][0] # Vijay  
puts students[1][2] # Mumbai  
puts students[2][1] # 20
```

Type 3 — Mixed Type Array

Ruby allows different data types in same array.

```
mixed = [1, "Vijay", 3.14, true, nil]  
  
puts mixed[0] # 1  
puts mixed[1] # Vijay  
puts mixed[2] # 3.14  
puts mixed[3] # true
```

Type 4 — Empty Array

Array with no elements — add later.

```
arr = [] # empty array  
arr << "Apple" # add element  
arr << "Mango"  
puts arr.inspect # ["Apple", "Mango"]
```

Array Operations — Table

Operation	Method	Example	Output
Add at end	push or <<	arr.push("Grapes")	adds Grapes at end
Add at start	unshift	arr.unshift("Orange")	adds Orange at start
Remove last	pop	arr.pop	removes last element
Remove first	shift	arr.shift	removes first element
Remove specific	delete	arr.delete("Mango")	removes Mango
Total count	length	arr.length	5
Sort	sort	arr.sort	sorted array
Reverse	reverse	arr.reverse	reversed array
Minimum	min	arr.min	smallest value
Maximum	max	arr.max	largest value

6. Hashes in Ruby

What is a Hash?

A Hash is a collection of **key-value pairs**. Like an associative array in PHP. Think of it like a **dictionary** — word (key) and its meaning (value).

Key Points :

Hash stores data as **key => value** pairs

Keys are **unique** — no duplicate keys

Values can be **any data type**

Keys can be **strings or symbols**

Hash is like **associative array** in PHP

Faster to search than array

Uses **{ }** curly braces

Access value using **key** not index number.



6. Hashes in Ruby

Method 1 — using string keys

```
student = {  
  "name" => "Vijay",  
  "age"  => 21,  
  "city" => "Pune"  
}
```

Method 2 — using symbol keys

(more common in Ruby)

```
student = {  
  name: "Vijay",  
  age: 21,  
  city: "Pune"  
}
```

Accessing Hash Values :

```
student = {name: "Vijay", age: 21, city: "Pune"}
```

```
puts student[:name] # Vijay  
puts student[:age]  # 21  
puts student[:city] # Pune
```



7. Methods in Ruby

What is a Method?

A method is a **block of code** that performs a specific task.
Write once — use many times.

1. Basic Method

```
def greet  
  puts "Hello! Welcome to Ruby!"  
end
```

```
greet  # calling the method
```

Output:

Hello! Welcome to Ruby!

2. Method with Parameters

```
def greet_student(name)  
  puts "Hello #{name}!"  
end
```

```
greet_student("Vijay")  
greet_student("Rahul")
```

Output:

Hello Vijay!

Hello Rahul!



3. Method with Return Value

```
def add(a, b)
  sum = a + b
  return sum # explicit return
end
```

```
result = add(10, 20)
puts result # 30
```

4. Method — Implicit Return

Ruby automatically returns **last line value**.

No need to write return!

```
def add(a, b)
  a + b # last line — returned automatically
end
```

```
puts add(10, 20) # 30
```

5. Method with Default Parameters

```
def greet(name = "Student")
  puts "Hello #{name}!"
end
```

```
greet("Vijay") # Hello Vijay!
greet          # Hello Student! (uses default)
```



8. Classes in Ruby

What is a Class?

A class is a **blueprint** for creating objects.

It defines **properties (variables)** and **behaviors (methods)** of an object.

Key Points :

Class is defined using **class** keyword

Class name always starts with **Capital letter**

Object is created using **new** keyword

Methods inside class are called using **dot (.)** operator

Class ends with **end** keyword.

Basic Class :

```
class Student
  def greet
    puts "Hello I am a Student!"
  end
end
```

```
# Create object
s = Student.new
s.greet
```

Output:

Hello I am a Student!



9. Code Blocks and Iterators in Ruby

What is a Code Block?

A code block is a **chunk of code** that you pass to a method. It runs inside that method.

Two Ways to Write Blocks

1. Multi line — do...end

```
[1, 2, 3].each do |num|  
  puts num  
end
```

2. Single line — { }

```
[1, 2, 3].each { |num| puts num }
```

Both give same output:

```
1  
2  
3
```




9. Code Blocks and Iterators in Ruby

Iterators in Ruby :

Iterators are **built-in methods** that use blocks to loop through collections.

1. **each** — loop through array

```
fruits = ["Apple", "Mango", "Banana"]
```

```
fruits.each do |fruit|  
  puts fruit  
end
```

Output:

```
Apple  
Mango  
Banana
```

2. **times** — repeat N times

```
3.times do |i|  
  puts "Hello #{i}"  
end
```

Output:

```
Hello 0  
Hello 1  
Hello 2
```

9. Code Blocks and Iterators in Ruby

3. upto — count up

```
1.upto(5) do |i|  
  puts i  
end
```

Output:

1
2
3
4
5

4. downto — count down

```
5.downto(1) do |i|  
  puts i  
end
```

Output:

5
4
3
2
1

10. Pattern Matching in Ruby

What is Pattern Matching?

Pattern matching means **searching for a specific pattern** inside a string.

Ruby uses **Regular Expressions (Regex)** for pattern matching.

Same concept as PHP but Ruby has **cleaner syntax**.

Common Regex Patterns :

Pattern	Meaning	Example
/[0-9]/	Any digit	3, 7
/[a-z]/	Lowercase letter	a, b
/[A-Z]/	Uppercase letter	A, B
/\d/	Any digit (short)	3, 7
/\w/	Any word character	a, 3
/\s/	Whitespace	space
/^Hello/	Starts with Hello	Hello World
/World\$/	Ends with World	Hello World



11. Overview of Rails

What is Rails?

Rails (full name **Ruby on Rails**) is a **web application framework** written in Ruby.

It makes building web applications **faster and easier**.

Think of it like a **ready made structure** for building websites — you just fill in the parts.

Key Points :

Full name = **Ruby on Rails (RoR)**

Created in **2004**

Built on top of **Ruby** language.

Follows **MVC architecture** (Model View Controller)

Follows **Convention over Configuration** — less setup needed

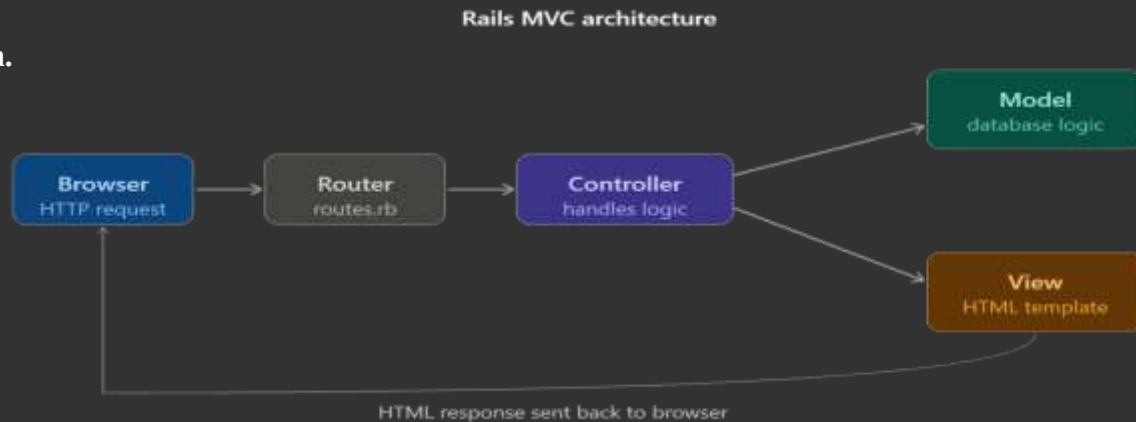
Used by **GitHub, Shopify, Twitter**

File extension = **.rb** (Ruby files)

11. Overview of Rails

MVC Architecture in Rails

Rails strictly follows **MVC pattern**.



1. Model

Handles **database** related work.

Talks to database — fetch, save, update data.

Example — `Student.all` fetches all students from DB.

2. View

Handles **what user sees** — HTML pages.

Contains HTML + Ruby code (`.html.erb` files).

Example — shows student list on webpage.

3. Controller

Handles **logic** — middle man between Model and View.

Gets data from Model, sends to View.

Example — `StudentsController` handles student related requests.

12. Document Requests in Rails

What is a Document Request?

When a user types a URL in browser, they are making a **document request**.
Rails handles this request and sends back the right page.

Key Points :

Every URL request goes through **Router first**

Router is defined in **config/routes.rb** file

Router decides **which controller and action** to call

Rails supports **GET, POST, PUT, DELETE** request types

Rails uses **RESTful routing** by default

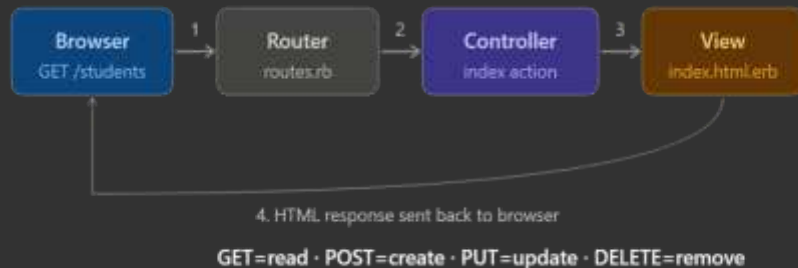
Each request maps to a **controller action**

Controller action then renders a **view**

12. Document Requests in Rails

How Document Request Works :

Rails document request flow



1. Browser

User types URL in browser — it sends a **HTTP GET/POST request** to the Rails server.

HTTP Methods

GET → read/fetch data

POST → create/submit data

PUT → update existing data

DELETE → remove data

2. Router (routes.rb)

Receives the request and decides **which controller and action** to call based on URL.

3. Controller

Executes the **action method** — gets data from Model and prepares it for View.

4. View (index.html.erb)

Takes data from Controller and **generates HTML page** to send back to browser.

5. HTML Response

Final **HTML page** is sent back to browser and user sees the output on screen.

13. Processing Forms in Rails

What is Form Processing in Rails?

Form processing means **collecting data from HTML form** and saving it to database using Rails.

Key Points :

Rails uses **form_tag** or **form_for** helper to create forms

form_for is linked to a **model object**

form_tag is used for **simple forms**

Form data submitted via **POST method**

Controller receives data in **params** hash

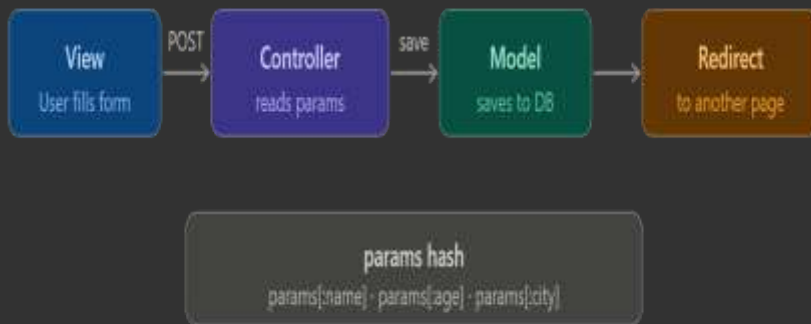
`params[:field_name]` → access form data

Data saved to database using **Model**

Rails has **strong parameters** for security

After saving — usually **redirect** to another page.

Rails form processing flow





14. Rails Applications and Databases

What is a Rails Application?

A Rails application is a **complete web application** built using Ruby on Rails framework. It has everything ready — database, routing, controllers, views — all organized in one folder structure.

How to Create & Run Rails App :

Step 1 — Create new app

```
rails new studentapp
```

Step 2 — Go inside app folder

```
cd studentapp
```

Step 3 — Generate model

```
rails generate model Student name:string age:integer
```

Step 4 — Run migration (create table)

```
rake db:migrate
```

Step 5 — Generate controller

```
rails generate controller Students index
```

Step 6 — Start server

```
rails server
```

Now open browser → <http://localhost:3000>

14. Rails Applications and Databases

What is Rails with Database?

Rails connects to database to **store, fetch, update and delete** data.

Rails uses **ActiveRecord** to talk to database — no need to write SQL manually!

Think of ActiveRecord like a **translator** — you write Ruby, it converts to SQL automatically.

ORM Concept — ActiveRecord :

Rails uses **ActiveRecord** as database library

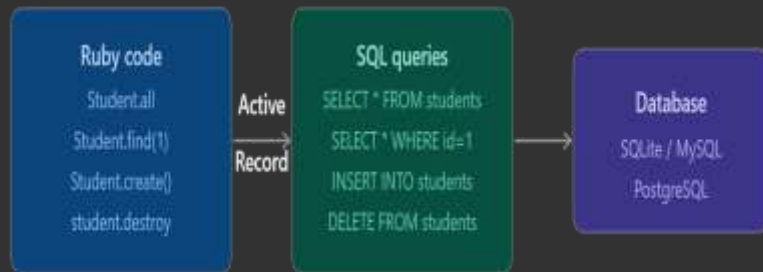
ActiveRecord = **ORM (Object Relational Mapping)**

ORM = converts Ruby objects to database rows automatically

Default database = **SQLite** (for development)

Can use **MySQL, PostgreSQL** also

ActiveRecord — ORM in Rails



ActiveRecord converts Ruby → SQL automatically. No need to write SQL

15. Layouts in Rails

What is a Layout in Rails?

A Layout is a **common template** that wraps around all your pages.

It contains common parts like **header, footer, navigation** that appear on every page.

How Layout Works:

1. application.html.erb (Layout)

This is the **main template file**

— it wraps around every page of the application.

2. Header / Navigation

Written once in layout

— appears on **every page** automatically without repeating.

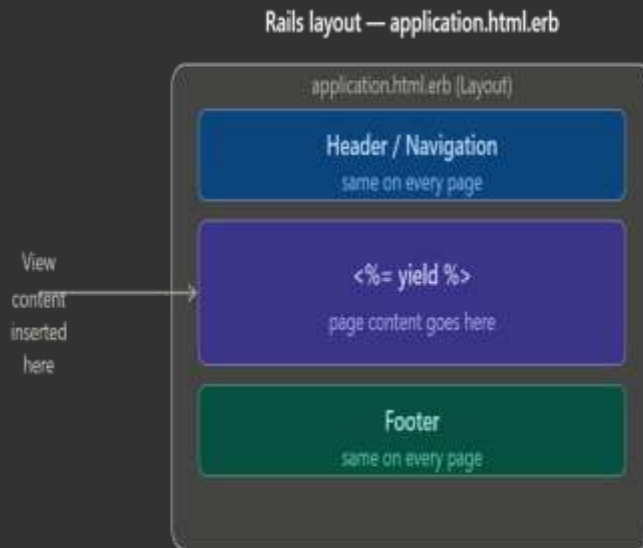
3. `<%= yield %>`

This is a **placeholder**

— each page's own content gets inserted here automatically.

4. Footer

Written once in layout — appears at **bottom of every page** automatically.



16. Rails with Ajax

What is Ajax in Rails?

Ajax stands for **Asynchronous JavaScript and XML**.

It allows web page to **update part of page** without reloading the whole page.

Think of it like **Facebook likes** — when you click like, only like count updates, whole page does not reload.

How Ajax Works in Rails :

1. User clicks button / submits form



2. JavaScript sends request to server
(background)



3. Controller processes request



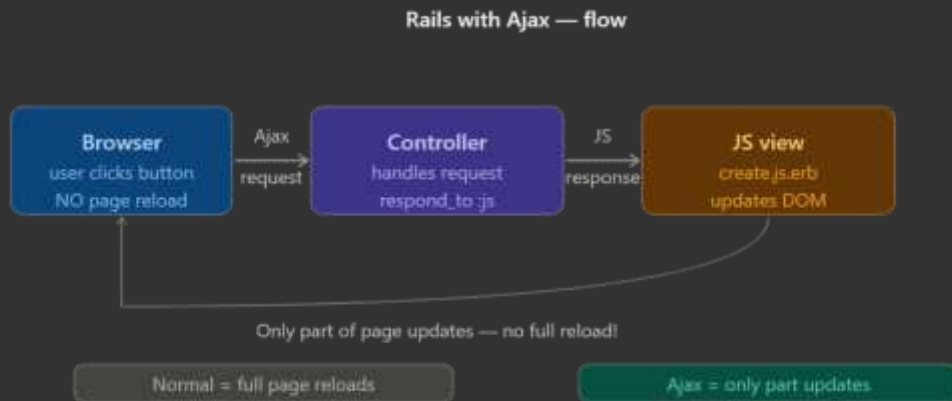
4. Server sends back JS response



5. JavaScript updates only specific part of page



6. User sees update — NO page reload!



17. Introduction to EJB

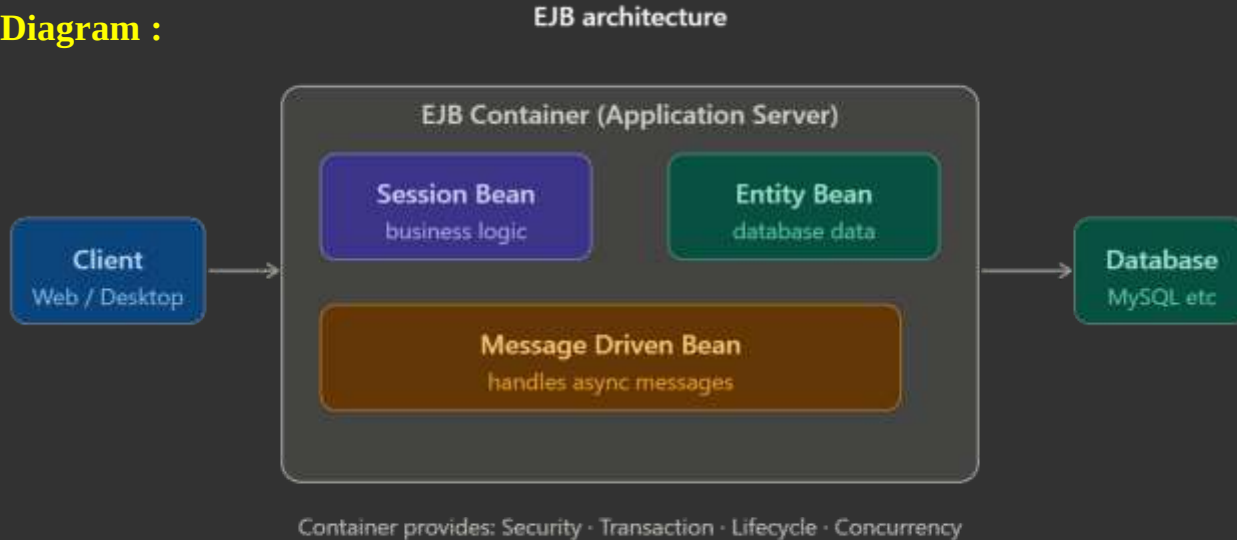
What is EJB?

EJB stands for **Enterprise Java Beans**.

It is a **server side component** used to build large scale, secure enterprise applications in Java.

Think of it like **ready made building blocks** for big enterprise Java applications.

EJB Architecture Diagram :



17. Introduction to EJB

EJB Architecture :

1. Client (Web / Desktop)

The user or application that **sends requests** to the EJB container to perform business tasks.

2. EJB Container (Application Server)

The **heart of EJB** — it manages all beans and provides services like security, transactions and lifecycle automatically.

3. Session Bean

Contains **business logic** — performs tasks like calculations, processing and operations for the client.

4. Entity Bean

Represents **data stored in database** — each entity bean maps to one table or one row.

5. Message Driven Bean

Handles **asynchronous messages** — listens to a message queue and processes messages in background.

6. Database

Stores the actual **data permanently** — Entity beans and Session beans read/write data here.



Thank You!



Like, Share & Subscribe

<https://youtube.com/@thevijayacademy?si=uYwIDuAxyZfLsiei>

Also join Telegram channel for updates and notes :

https://t.me/the_vijay_academy